

FAST NATIONAL UNIVERSITY OF COMPUTER AND EMERGING SCIENCES

Lahore Campus

PHASE 1

StackTrack AI

Project Proposal

Course Name	DS3001 — Data Analysis and Visualization
Instructor	Mr. Amir Iqbal
Section	BDS-5B
Submitted By	Muhammad Ibrahim (24L-2602) Safee Akmal (23L-2556)

DEPARTMENT OF DATA SCIENCE

Table of Contents

1. Domain & Source Identification	4
1.1 Project Concept	4
1.2 Data Sources	4
1.3 Exact Source References	4
2. Ingestion Pattern	4
2.1 Full Load (Historical Baseline)	4
2.2 Incremental Load	5
3. Data Samples & Volume	5
3.1 Source 1 — GH Archive (GitHub Events)	5
3.2 Source 2 — Hugging Face Hub API	7
3.3 Source 3 — PyPI Stats API (Planned Extension)	8
3.4 Infrastructure Capacity & FinOps	9
3.5 Volume Summary & Viability Assessment	9
3.6 AI Technology Universe (Anchor Repositories)	10
4. Security & Compliance	12
4.1 PII and Sensitive Data Analysis	12
4.2 Handling Strategy: Three-Tier Privacy Model	13
4.3 PII Summary Table	14
5. High-Level Medallion Data Modeling	14
5.1 Bronze Layer (Raw)	14
5.2 Silver Layer (Cleansed)	14
5.3 Gold Layer (Business-Ready)	15
6. Business Intelligence & Dashboards	15
6.1 Dashboard Pages & Business Questions	15
6.2 Page 1 — Executive Overview	16
6.3 Page 2 — Technology Deep Dive	16
6.4 Page 3 — Ecosystem Comparison	17
6.5 Page 4 — Cross-Source Lag Analysis	18

6.6 Page 5 — Pipeline Health Monitor	19
6.7 Data Flow — From Gold to Dashboard	19
6.8 Dashboard Summary	20
7. Engineering Setup & FinOps	21
7.1 Version Control	21
7.2 Deployment Platform	21
7.3 Recommended Platform Stack for This Project	22
7.4 Cost Guardrails & Budget Discipline	22
7.5 Estimated Monthly Cost at Scale	23
8. Definition of Done (Phase 1 Success Criteria)	23

Phase 1 Project Proposal: StackTrack AI

1. Domain & Source Identification

1.1 Project Concept

The chosen domain is **Open-Source Repository and Ecosystem Analytics**, specifically focusing on the AI technology stack. The project, titled "StackTrack AI," tracks how artificial intelligence technologies evolve across software engineering activity (code contributions) and AI-model ecosystem adoption over time.

1.2 Data Sources

We will utilize two primary data sources representing different facets of the AI ecosystem:

1. **Software Engineering Activity:** GitHub via the **GH Archive** (public event stream of GitHub activity).
2. **AI Model Ecosystem Activity:** **Hugging Face Hub API** (model repository metadata, downloads, and adoption metrics).

1.3 Exact Source References

- **GH Archive:** `https://data.gharchive.org/YYYY-MM-DD-H.json.gz` (Automated hourly data dumps).
 - **Hugging Face Hub API:** `https://huggingface.co/api/models` (REST API for model metadata).
-

2. Ingestion Pattern

The selected data sources support both required ingestion patterns:

2.1 Full Load (Historical Baseline)

- **GH Archive:** We can perform a full load by downloading historical hourly JSON archives for a specific baseline period (e.g., all hours in a given month or year) to establish our initial historical warehouse.
- **Hugging Face:** We will perform an initial full extraction of the metadata for a defined "universe" of AI models (e.g., models tagged with specific frameworks or tasks) to create our baseline snapshot.

2.2 Incremental Load

- **GH Archive:** The pipeline will periodically fetch newly published hourly archives (e.g., `2026-10-01-15.json.gz`). This append-only pattern is naturally incremental.
- **Hugging Face:** We will take periodic (e.g., daily) snapshots via the API. Our incremental logic will use a "snapshot-diff" strategy, comparing the new snapshot against the previous one to identify `NEW`, `UPDATED`, `UNCHANGED`, or `MISSING` models.

3. Data Samples & Volume

Sample raw data files for both Full Load and Incremental Load payloads will be committed to the `data/samples/` directory of the GitHub repository as part of Phase 1 delivery.

This section documents the complete, research-backed data volume analysis for every source in the project. All figures are derived from official source documentation and published community analyses. Sources are cited inline.

3.1 Source 1 — GH Archive (GitHub Events)

Official source: <https://www.gharchive.org/>
Data format: Hourly `.json.gz` files
URL pattern: `https://data.gharchive.org/YYYY-MM-DD-H.json.gz`

Raw Source Volume (Unfiltered)

GH Archive has been recording the full public GitHub event stream since 2011. The dataset has grown significantly as GitHub's global usage has scaled.

Metric	Value	Source
Total records (all years, 2011–present)	11+ billion events	ClickHouse GitHub analysis
Total dataset size (all years, compressed)	~17+ TB	Community analysis
Estimated size per year (recent years)	~7 TB compressed	YouTube/community estimate
Typical hourly file size (2024–2026)	100 MB – 500 MB per <code>.json.gz</code>	aravpanwar.com
Files per day	24 (one per hour)	gharchive.org
Estimated 1 day (raw, compressed)	2.4 GB – 12 GB	Calculated
Estimated 1 month (raw, compressed)	~72 GB – 360 GB	Calculated
Estimated 1 year (raw, compressed)	~7 TB	Community estimate

Known Data Quality Issue (2025+): Since mid-2025, GH Archive event feeds have become heavily dominated by `PushEvent` records. Other event types (stars, forks, issues, pull requests) are significantly undercounted or missing from recent archives. This limitation will be documented in the project's source quality metrics and the pipeline's completeness monitoring will flag these anomalies. Source: ClickHouse GitHub analysis.

Our Project's Filtered Volume

Our pipeline does **not** process all of GitHub. We apply an early filter at the Bronze → Silver boundary, retaining only events from our curated universe of ~50–100 AI technology repositories (e.g., `pytorch/pytorch`, `huggingface/transformers`, `langchain-ai/langchain`).

These repositories represent approximately **0.001% – 0.01%** of all GitHub public activity.

Metric	Raw (All GitHub)	Our Filtered Volume
Hourly Bronze file	100–500 MB	~100 KB – 5 MB
Daily Bronze ingest	2.4–12 GB	~2.4 MB – 120 MB
Monthly Bronze total	72–360 GB	~72 MB – 3.6 GB
Daily Silver (Parquet, compressed)	—	~1 MB – 50 MB
Monthly Silver total	—	~30 MB – 1.5 GB
Gold daily fact table (aggregated)	—	< 10 MB/day

Full Load scope for this project: 3 months of historical data (January–March 2026)

Estimated full load Bronze size (filtered): ~216 MB – 10.8 GB

Estimated full load Silver size (filtered, Parquet): ~90 MB – 4.5 GB

Incremental Load

Metric	Value
Frequency	Hourly (each new <code>.json.gz</code> file)
Incremental mechanism	Checkpoint-based: <code>last_processed_hour</code> → next hour
Incremental Bronze size per run	~100 KB – 5 MB
Incremental Silver delta per day	~1 MB – 50 MB
Idempotency	Yes — partition key + ingestion manifest prevents duplicates

Rate Limits & Throttling Strategy

- **Limit:** GH Archive does not enforce strict API limits as it serves static `.json.gz` files over HTTP.
- **Handling:** To prevent IP blocking and respect source bandwidth, the pipeline will enforce a short delay (e.g., 1-2 seconds) between sequential hour downloads during full backfills.

3.2 Source 2 — Hugging Face Hub API

Official API: <https://huggingface.co/api/models>

Python client: https://huggingface.co/docs/huggingface_hub/

Hub stats: <https://huggingface.co/spaces/cfahlgren1/hub-stats>

Raw Source Volume (Unfiltered)

Metric	Value	Source
Total public models (mid-2026)	~3 million	huggingface.co blog
Growth rate (2025–2026)	2nd million added in 335 days (65% faster than the 1st)	aiworld.eu
Estimated new models globally per day	~2,000 – 5,000	Calculated from growth rate
Metadata per model (JSON, standard fields)	~1 KB – 10 KB	HF API documentation
Full snapshot of all 3M models	~3 GB – 30 GB	Calculated
Daily global incremental diff	~2 MB – 50 MB	Calculated

Our Project's Filtered Volume

Our pipeline snapshots metadata only for models belonging to our curated AI technology universe (e.g., models tagged `pytorch`, `transformers`, `diffusers`, `langchain`). This covers thousands — not millions — of models.

Metric	Raw (All HF Models)	Our Filtered Volume
Full snapshot	3 GB – 30 GB	~5 MB – 50 MB
Daily incremental diff	2–50 MB	< 1 MB
New models per day (our universe)	2,000–5,000 globally	~10–100 relevant models/day
Silver (Parquet, normalized snapshots)	—	< 20 MB per snapshot
Gold fact (daily adoption metrics)	—	< 5 MB per day

Full Load: Initial API snapshot of all models matching our technology universe tags.

Estimated full load size: 5–50 MB (trivially manageable).

Incremental Load

Metric	Value
Frequency	Daily snapshot

Metric	Value
Incremental mechanism	Snapshot-diff: compare <code>lastModified</code> + field hash between Snapshot N and Snapshot N+1
Statuses produced	<code>NEW</code> , <code>UPDATED</code> , <code>UNCHANGED</code> , <code>MISSING</code>
Incremental Bronze size per run	< 1 MB (our filtered universe)
Note on MISSING records	A missing model may indicate deletion, privacy change, or API filtering — not confirmed deletion. History is preserved in Bronze.

Rate Limits & Throttling Strategy

- **Limit:** The Hugging Face Hub API enforces rate limits evaluated over 5-minute windows. Exceeding limits returns a `429 Too Many Requests` error.
- **Handling:**
- **Authentication:** All requests will pass an authenticated `HF_TOKEN` from Databricks Secrets, which significantly increases the baseline rate limit.
- **Backoff:** The PySpark ingestion job will implement an exponential backoff with jitter retry strategy to automatically sleep and retry if a `429` response is encountered during a snapshot run.

3.3 Source 3 — PyPI Stats API (*Planned Extension*)

Official API: <https://pypistats.org/api>

BigQuery public dataset: `bigquery-public-data.pypi.file_downloads`

Raw Source Volume

Metric	Value	Source
Full PyPI BigQuery dataset (all packages, all time)	~440 TB	BigQuery public dataset docs
API history retention window	180 days only	pypistats.org
Update frequency	Daily (once per day)	pypistats.org
Per-package, per-day API response	~1–2 KB (JSON)	pypistats.org
BigQuery free query tier	1 TB/month	Google Cloud free tier

Important Constraint: The `pypistats.org` API only retains 180 days of time-series data. The project **will not claim** to have unlimited historical PyPI data. The pipeline will document this window explicitly and the ingestion manifest will record the earliest available date.

Our Project's Filtered Volume

Our pipeline collects daily download statistics only for the ~50–100 Python packages in our technology universe (e.g., `torch`, `transformers`, `langchain`, `diffusers`).

Metric	Raw (All PyPI Packages)	Our Filtered Volume
Full historical load (180 days)	440 TB (BigQuery)	< 10 MB (our packages only)
Daily incremental per run	N/A	< 1 MB
Bronze storage (180-day baseline)	—	< 10 MB
Silver Parquet (180-day baseline)	—	< 5 MB
Gold daily fact	—	< 1 MB/day

3.4 Infrastructure Capacity & FinOps

Platform: Databricks Free Edition (replaced Community Edition in 2025)

Reference: <https://www.databricks.com/product/free-edition>

Constraint	Free Edition Limit	Impact on Our Project
Compute type	Serverless only (fair usage quota)	Sufficient for our filtered data volumes
SQL Warehouse	1 × 2X-Small cluster	Adequate for Gold layer queries
Concurrent job tasks	5 max	Our daily pipeline fits within this
GPU compute	Not available	Not required — we process metadata, not model weights
Custom storage	Not available (no external S3/ADLS)	Data stored in Databricks managed storage
Languages supported	Python, SQL	Our stack is Python + PySpark
BigQuery free tier	1 TB queries/month	Our PyPI queries use < 1 GB/month

3.5 Volume Summary & Viability Assessment

The critical insight of this section is that **the raw source volumes are enormous, but our filtered project volumes are small and fully manageable on free-tier infrastructure.**

Source	Raw Volume (Unfiltered)	Our Filtered Volume	Infrastructure Fit
GH Archive — Full Load (3 months)	216 GB – 1 TB	~216 MB – 10.8 GB	Manageable
GH Archive — Daily Incremental	2.4–12 GB/day	~2.4–120 MB/day	Very manageable
Hugging Face — Full Snapshot	3–30 GB	~5–50 MB	Trivially small

Source	Raw Volume (Unfiltered)	Our Filtered Volume	Infrastructure Fit
Hugging Face — Daily Incremental	2–50 MB/day	< 1 MB/day	Trivially small
PyPI — Full Load (180-day window)	440 TB (BigQuery)	< 10 MB	Trivially small
PyPI — Daily Incremental	N/A	< 1 MB/day	Trivially small

Why processing real, large-scale data still qualifies as a Spark project:

The GH Archive source, even after filtering for our 50–100 repositories, may produce Bronze files of up to 10.8 GB for a 3-month backfill. Parsing hourly JSON.GZ files, applying schema enforcement, flattening nested structures, deduplicating records, and writing partitioned Parquet output across this volume is a legitimate distributed Spark workload — it is simply scoped to be Databricks Free Edition compatible.

Volume mitigation strategy:

1. **Bronze-to-Silver filter:** Ingest the raw firehose into Bronze, but strictly apply the technology universe filter during the PySpark job that writes to Silver.
2. **Bronze retention policy:** Enforce a strict 7-day retention policy on the Bronze layer to manage storage costs, as raw data can be redownloaded from GH Archive if needed.
3. **Date-bounded backfill:** Limit the initial full load to 3 months (not 12), expandable incrementally.
4. **Parquet + Snappy compression:** Reduces Silver/Gold storage by 60–80% compared to raw JSON.
5. **Partition pruning:** Gold queries will use `year/month` partitions to avoid full table scans.
6. **BigQuery for PyPI:** Use BigQuery's 1 TB/month free tier for PyPI rather than storing a local copy of the 440 TB raw dataset.

3.6 AI Technology Universe (Anchor Repositories)

To ensure the scope remains manageable and highly relevant, the pipeline will filter the raw GH Archive and Hugging Face firehoses down to a curated "universe" of approximately 45–50 core AI technologies. The remaining repositories (to reach the ~100 limit) can be dynamically loaded via a configuration file as the ecosystem evolves.

The initial anchor list explicitly tracks:

1. Foundational AI Frameworks

- `pytorch/pytorch` (Meta)
- `tensorflow/tensorflow` (Google)
- `google/jax` (Google)
- `keras-team/keras` (Google/Keras)
- `microsoft/onnxruntime` (Microsoft)

2. Model Tooling & Fine-Tuning

- `huggingface/transformers` (Hugging Face)
- `huggingface/peft` (Hugging Face)

-
- `huggingface/accelerate` (Hugging Face)
 - `huggingface/trl` (Hugging Face)
 - `huggingface/datasets` (Hugging Face)

3. Inference, Serving & Execution

- `vllm-project/vllm` (vLLM)
- `huggingface/text-generation-inference` (Hugging Face)
- `ggerganov/llama.cpp` (Community)
- `ollama/ollama` (Ollama)
- `microsoft/DeepSpeed` (Microsoft)
- `triton-inference-server/server` (Nvidia)

4. Agent & Application Frameworks

- `langchain-ai/langchain` (LangChain)
- `run-llama/llama_index` (LlamaIndex)
- `microsoft/autogen` (Microsoft)
- `joaomdmoura/crewAI` (CrewAI)
- `deepset-ai/haystack` (Deepset)

5. Generative AI & Vision

- `huggingface/diffusers` (Hugging Face)
- `AUTOMATIC1111/stable-diffusion-webui` (Community)
- `comfyanonymous/ComfyUI` (Community)
- `ultralytics/ultralytics` (Ultralytics)
- `open-mmlab/mmdetection` (OpenMMLab)

6. Distributed & MLOps Infrastructure

- `ray-project/ray` (Anyscale)
- `mlflow/mlflow` (Databricks)
- `iterative/dvc` (Iterative)
- `bentoml/BentoML` (BentoML)
- `kubeflow/kubeflow` (Google)

7. Core Open-Weight Model Codebases

- `meta-llama/llama` (Meta)
 - `mistralai/mistral-src` (Mistral AI)
 - `QwenLM/Qwen` (Alibaba)
 - `EleutherAI/gpt-neox` (EleutherAI)
 - `nomic-ai/gpt4all` (Nomic AI)
-

4. Security & Compliance

4.1 PII and Sensitive Data Analysis

A careful field-level analysis was performed on each data source. The findings are documented below.

Source 1 — GH Archive (GitHub Events)

GH Archive contains two distinct layers of PII:

Layer 1 — Event metadata (present in every event type):

Field	Description	PII Classification
<code>actor.login</code>	GitHub username of the person who performed the action	Quasi-PII — public handle, uniquely identifies a real person
<code>actor.display_login</code>	Display name, usually identical to <code>actor.login</code>	Quasi-PII
<code>actor.url</code>	GitHub API URL linking to the user's profile	Quasi-PII — resolves to a real identity
<code>actor.avatar_url</code>	URL to the user's profile picture	Quasi-PII

Layer 2 — `PushEvent` commit payload (present only in push events):

Field	Description	PII Classification
<code>payload.commits[].author.email</code>	Git commit author email embedded in Git commit metadata	Hard PII — a real, personal email address under GDPR
<code>payload.commits[].author.name</code>	Committer's display name as configured in their local Git client	Hard PII — may be a real full name

Note: Under GDPR, email addresses are classified as personal data. When a pipeline ingests GH Archive data, the team becomes a "data processor" for this information. Although the data is publicly accessible on GitHub, this does not remove the obligation to minimise, protect, and handle it responsibly. GH Archive itself has historically taken steps such as hashing emails to address this issue.

Source 2 — Hugging Face Hub API

Field	Description	PII Classification
<code>author</code>	HF username or organization namespace	Quasi-PII — a public platform handle, may map to a real name

The Hugging Face model metadata API does **not** expose email addresses, IP addresses, or any other hard PII in its public endpoints. The `author` field is a deliberately public identifier equivalent to a GitHub username.

Source 3 — PyPI Stats API

Field	Description	PII Classification
All fields	Aggregate download counts, package name, date, Python version, OS	No PII whatsoever

PyPI download statistics are fully aggregated and anonymised by the Python Software Foundation (PSF). The PSF explicitly excludes individual user identifiers from the public BigQuery dataset and the APIs built on top of it. No linking of a download to an individual is possible through this source.

4.2 Handling Strategy: Three-Tier Privacy Model

We adopt the industry-standard three-tier privacy pattern, applied across the Medallion layers:

Tier 1 — Bronze (Raw, Access-Controlled)

- Raw source data is stored **as-is**, preserving all original fields including PII.
- Bronze storage will be access-controlled and treated as a restricted zone, never used as a direct query or dashboard source.
- **Rationale:** Immutability of Bronze is a core architectural principle. Destroying data at ingestion prevents reprocessing from raw if the Silver transformation logic changes. Protection is enforced via access controls, not deletion.

Tier 2 — Silver (Hashed / Masked)

The following transformations are applied strictly during the Bronze → Silver Spark job:

Source Field	Silver Treatment	Reason
<code>actor.login</code> (GH)	SHA-256 hashed → <code>actor_id_hash</code>	Enables unique contributor counts without storing raw usernames
<code>actor.display_login</code> (GH)	Dropped	Duplicate of <code>actor.login</code> , not required for analysis
<code>actor.url</code> (GH)	Dropped	Not required for analysis
<code>actor.avatar_url</code> (GH)	Dropped	Not required for analysis
<code>payload.commits[].author.email</code> (GH)	Dropped immediately	Hard PII — never propagated beyond Bronze under any circumstances
<code>payload.commits[].author.name</code> (GH)	Dropped immediately	Hard PII — never propagated beyond Bronze
<code>author</code> (HF)	Retained as-is	Public platform identifier used for model grouping; no masking required
All PyPI fields	Retained as-is	No PII present

Why hash `actor.login` instead of dropping it entirely? Hashing allows the pipeline to compute `COUNT(DISTINCT actor_id_hash)` per technology per day — a key engineering-activity metric — without ever storing or exposing the raw username in Silver or Gold.

Tier 3 — Gold (Aggregated Only)

- The Gold layer contains **no individual-level identifiers** of any kind.
- All contributor metrics are expressed as aggregate counts (e.g., `unique_contributors_count`) derived from the hashed Silver IDs.
- No raw or hashed username appears in any Gold fact or dimension table.
- Dashboard users see only technology-level and time-period-level aggregations.

4.3 PII Summary Table

Source	Hard PII Present?	Quasi-PII Present?	Strategy
GH Archive — event metadata	No	Yes (<code>actor.login</code> , etc.)	Hash in Silver, aggregate only in Gold
GH Archive — PushEvent commits	Yes (emails, names)	—	Dropped at Bronze → Silver boundary
Hugging Face Hub API	No	Yes (<code>author</code> username)	Retain in Silver (public identifier), aggregate only in Gold
PyPI Stats API	No PII at all	—	No action required

5. High-Level Medallion Data Modeling

5.1 Bronze Layer (Raw)

Raw JSON files from GH Archive and Hugging Face API responses will be stored immutably in partitioned directories (e.g., partitioned by `year/month/day/hour` for GitHub and `snapshot_date` for Hugging Face). No destructive transformations will occur here.

5.2 Silver Layer (Cleansed)

Spark will process the Bronze data into Parquet format. Major steps include:

- Schema enforcement and casting strings to appropriate timestamp or numeric types.
- Flattening deeply nested JSON structures (e.g., extracting specific fields from GitHub event payloads).

- Dropping/hashing PII.
- Filtering the massive GitHub firehose down to only repositories relevant to our defined "AI Technology Universe" using a configuration mapping.

Planned Silver Model: `silver_github_events` (cleaned event stream), `silver_hf_models` (normalized model metadata).

5.3 Gold Layer (Business-Ready)

The Gold layer will utilize a dimensional (star) schema designed for analytical queries.

- **Dimensions:** `dim_date`, `dim_technology` (a conformed dimension resolving identities across GitHub and HF), `dim_category`.
- **Facts:**
 - `fact_github_activity` (Aggregated daily counts of PRs, commits, issues per technology).
 - `fact_hf_adoption` (Daily snapshot of model downloads, likes, and creation counts per technology).
 - `fact_technology_daily_snapshot` (Daily composite metrics per repository to feed radar and trend charts).
 - `fact_hf_model_activity` (Tracks Hugging Face specific model metrics over time).

6. Business Intelligence & Dashboards

The dashboard is the final consumer of the Gold layer. Every visual must be sourced exclusively from Gold fact and dimension tables — **never from raw Bronze or Silver data directly.**

The dashboard is organized into **5 pages**, each answering a distinct analytical question about the AI ecosystem.

6.1 Dashboard Pages & Business Questions

Page	Core Question Answered
Page 1: Executive Overview	What is the current health and activity level of the AI technology stack?
Page 2: Technology Deep Dive	How has a specific technology evolved over time across all ecosystem layers?
Page 3: Ecosystem Comparison	How do AI technologies compare against each other in engineering and adoption?
Page 4: Cross-Source Lag Analysis	Does engineering activity precede model ecosystem adoption, and by how long?
Page 5: Pipeline Health	Is the data pipeline running correctly and delivering fresh, complete data?

6.2 Page 1 — Executive Overview

Purpose: A high-level summary visible at a glance. No filters required.

Chart 1 — AI Stack Activity Heatmap

- **Chart type:** Calendar Heatmap (colour intensity = daily event count)
- **Gold table:** `fact_github_activity`
- **Fields used:** `date_key`, `event_count` (summed across all technologies)
- **Business question:** On which days was the AI ecosystem most and least active?
- **Insight delivered:** Identifies weekends, holidays, and major release events that caused spikes or drops in engineering activity across the entire monitored AI stack.

Chart 2 — Technology Category Breakdown (Current Period)

- **Chart type:** Treemap
- **Gold tables:** `fact_github_activity` JOIN `dim_technology` JOIN `dim_category`
- **Fields used:** `category`, `subcategory`, `event_count`
- **Business question:** Which AI technology categories dominate current engineering activity?
- **Insight delivered:** Shows whether activity is concentrated in Frameworks, LLM Tooling, Inference/Serving, or Agent frameworks — and how that proportion changes over time.

Chart 3 — Top 10 Technologies by Combined Ecosystem Score

- **Chart type:** Horizontal Bar Chart (sorted descending)
- **Gold table:** `fact_technology_daily_snapshot`
- **Fields used:** `technology_name`, composite score derived from `github_events` + `hf_model_growth` (normalized)
- **Business question:** Which technologies are most active across all ecosystem layers right now?
- **Insight delivered:** A normalized ranking that prevents any single source (e.g., raw download counts) from dominating — a technology must show activity across multiple layers to rank highly.

6.3 Page 2 — Technology Deep Dive

Purpose: Select one technology and inspect its complete history across all sources.

Primary filter: `dim_technology.technology_name` (single-select dropdown)

Secondary filter: Date range selector

Chart 4 — Dual-Axis Engineering vs Adoption Timeline

- **Chart type:** Dual-Axis Line Chart (left Y: GitHub events; right Y: HF model downloads)
- **Gold tables:** `fact_github_activity` + `fact_hf_adoption` JOIN `dim_date` JOIN `dim_technology`
- **Fields used:** `date`, `event_count`, `downloads` (monthly aggregated)

- **Business question:** Does engineering activity and model ecosystem adoption for this technology move together, diverge, or is one leading the other?
- **Insight delivered:** The key cross-source comparison. For example, PyTorch may show sustained engineering activity while Hugging Face downloads spike months later when derivative fine-tuned models proliferate.

Chart 5 — Engineering Activity Breakdown (Stacked Area)

- **Chart type:** Stacked Area Chart
- **Gold table:** `fact_github_activity`
- **Fields used:** `date_key`, `event_type`, `event_count`
- **Business question:** What types of engineering actions are contributors performing on this technology over time?
- **Insight delivered:** Shows the composition of activity (PushEvents, PRs, Issues, Releases, Forks) and whether the community is shifting from active development (pushes/PRs) to maintenance (issues) or outreach (forks/stars).

Chart 6 — HF Model Growth Waterfall

- **Chart type:** Waterfall / Running Total Line Chart
- **Gold table:** `fact_hf_model_activity`
- **Fields used:** `date_key`, `new_models`, `modified_models`, `missing_models`
- **Business question:** How is the number of Hugging Face models built on top of this technology growing, and are models being retired?
- **Insight delivered:** Distinguishes between net model creation (actual ecosystem growth) and gross activity (which inflates if models are frequently updated but not newly created).

6.4 Page 3 — Ecosystem Comparison

Purpose: Compare multiple technologies side by side.

Primary filter: `dim_technology.technology_name` (multi-select, max 6)

Secondary filter: `dim_category.category` (filter by category)

Chart 7 — Technology Lifecycle Quadrant

- **Chart type:** Scatter Plot (bubble chart)
- **Gold table:** `fact_technology_daily_snapshot`
- **Fields used:** `growth_rate_90d` (X-axis), `activity_30d` (Y-axis), bubble size = `hf_model_count`
- **Quadrant labels:**
 - Top-right: **Rising** (high activity + high growth)
 - Top-left: **Mature** (high activity + slowing growth)
 - Bottom-right: **Emerging** (low activity + high growth)

- Bottom-left: **Declining** (low activity + low growth)
- **Business question:** At what lifecycle stage is each AI technology?
- **Insight delivered:** The single most powerful analytical view. Technologies move between quadrants over time and a time-lapse animation can show the full lifecycle arc.

Chart 8 — Cross-Technology GitHub Activity Race Chart

- **Chart type:** Animated Bar Race Chart (or static grouped bar for static reports)
- **Gold table:** `fact_github_activity` JOIN `dim_technology`
- **Fields used:** `date_key` (monthly), `technology_name`, `event_count`
- **Business question:** How has the relative ranking of AI technologies by engineering activity changed month by month?
- **Insight delivered:** Shows which technologies have overtaken others — e.g., did `vllm` overtake `TGI` in engineering activity in 2025?

Chart 9 — Normalized Radar / Spider Chart

- **Chart type:** Radar Chart (one axis per ecosystem dimension)
- **Gold tables:** `fact_github_activity` + `fact_hf_adoption` JOIN `dim_technology`
- **Axes:** GitHub Events, Unique Contributors, HF Model Count, HF Downloads
- **Business question:** How balanced is a technology's ecosystem across all four measured dimensions?
- **Insight delivered:** A technology strong only in downloads but weak in contributors and HF models may have fragile adoption. A balanced radar polygon indicates a healthy, multi-dimensional ecosystem.

6.5 Page 4 — Cross-Source Lag Analysis

Purpose: Investigate the temporal relationship between engineering activity and ecosystem adoption.

Chart 10 — Lead-Lag Correlation Timeline

- **Chart type:** Dual-Line Chart with Correlation Band
- **Gold tables:** `fact_github_activity` + `fact_hf_adoption` (both aggregated to weekly grain)
- **Fields used:** `week_start_date`, `github_events_normalized`, `hf_downloads_normalized`
- **Business question:** By how many weeks does a rise in GitHub engineering activity precede a rise in Hugging Face model downloads for the same technology?
- **Insight delivered:** If GitHub activity consistently peaks 4–8 weeks before HF downloads, this validates the research hypothesis that engineering activity is a leading indicator of ecosystem adoption — a novel and analytically strong finding.

Chart 11 — Event-Driven Spike Analysis

- **Chart type:** Annotated Line Chart
- **Gold table:** `fact_github_activity` + `fact_hf_model_activity`

- **Fields used:** `date`, `event_count`, `new_models`, annotated with `release_events` from `fact_github_activity` where `event_type = 'ReleaseEvent'`
 - **Business question:** Do major software releases (tagged via `ReleaseEvent`) trigger measurable spikes in HF model creation?
 - **Insight delivered:** Directly tests whether framework releases cause downstream model ecosystem expansion. Annotations mark release dates on the timeline, making the pattern visible without statistical analysis.
-

6.6 Page 5 — Pipeline Health Monitor

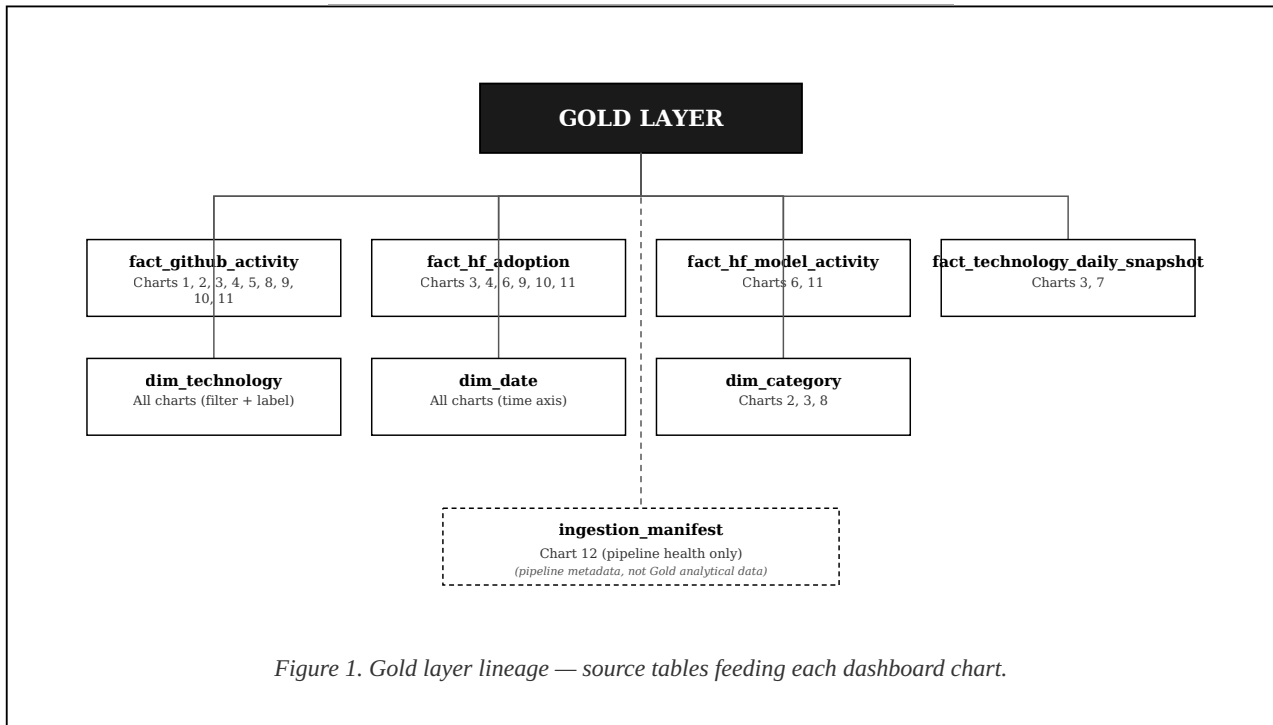
Purpose: Operational visibility into the data pipeline. The teacher asked specifically that the dashboard reflect real data engineering concerns, not just pretty charts.

Chart 12 — Data Freshness & Pipeline Status Table

- **Chart type:** Status Table with Conditional Formatting (green/amber/red)
 - **Source table:** `ingestion_manifest` (pipeline metadata, not Gold analytical data)
 - **Fields used:** `source`, `last_successful_partition`, `records_read`, `records_rejected`, `status`, `completed_at`
 - **Business question:** Is the pipeline running correctly? Is the data fresh? Are there data quality failures?
 - **Insight delivered:** Shows at a glance whether GH Archive, HF, and PyPI ingestion are healthy, stale, or failing. Rejected record counts flag data quality issues. This demonstrates that the project monitors its own reliability — a key data engineering maturity signal.
-

6.7 Data Flow — From Gold to Dashboard

Every chart above traces directly to a Gold layer table. No chart reads from Bronze or Silver.



6.8 Dashboard Summary

#	Chart Name	Type	Primary Gold Table	Business Question
1	AI Stack Activity Heatmap	Calendar Heatmap	<code>fact_github_activity</code>	When was the ecosystem most active?
2	Category Breakdown	Treemap	<code>fact_github_activity</code>	Which categories dominate?
3	Top 10 Technologies	Horizontal Bar	<code>fact_technology_daily_snapshot</code>	Who leads across all layers?
4	Engineering vs Adoption Timeline	Dual-Axis Line	<code>fact_github_activity</code> + <code>fact_hf_adoption</code>	Do they move together or diverge?
5	Engineering Activity Breakdown	Stacked Area	<code>fact_github_activity</code>	What kinds of actions are contributors taking?
6	HF Model Growth Waterfall	Waterfall / Line	<code>fact_hf_model_activity</code>	How fast is the model ecosystem growing?
7	Technology Lifecycle Quadrant	Bubble Scatter	<code>fact_technology_daily_snapshot</code>	What lifecycle stage is each technology in?

#	Chart Name	Type	Primary Gold Table	Business Question
8	Cross-Technology Race Chart	Bar Race	<code>fact_github_activity</code>	How has the activity ranking changed over time?
9	Ecosystem Radar	Radar / Spider	All fact tables	How balanced is each technology's ecosystem?
10	Lead-Lag Correlation	Dual-Line	<code>fact_github_activity</code> + <code>fact_hf_adoption</code>	How long before engineering activity drives adoption?
11	Event-Driven Spike Analysis	Annotated Line	<code>fact_github_activity</code> + <code>fact_hf_model_activity</code>	Do releases cause model ecosystem spikes?
12	Pipeline Health Monitor	Status Table	<code>ingestion_manifest</code>	Is the pipeline healthy and fresh?

7. Engineering Setup & FinOps

7.1 Version Control

The project code and sample data will be hosted on GitHub.

- **Repository Link:** *(Link will be provided upon repository creation)*
- **GitHub Education benefit:** Free private repositories, GitHub Copilot, and 80+ partner tool credits via the GitHub Student Developer Pack (<https://education.github.com/pack>)

7.2 Deployment Platform

This project is designed to run entirely within free-tier limits to ensure zero execution cost.

Databricks Free Edition

Property	Details
Program	Databricks Free Edition
URL	https://www.databricks.com/learn/free-edition
Cost	Free — no credit card required
Compute	Serverless only, fair usage quota with daily reset

Property	Details
SQL Warehouse	1 × 2X-Small (adequate for Gold analytical queries)
Concurrent jobs	5 max
GPU	Not available (not needed — we process metadata, not weights)
Languages	Python, SQL
Fit for this project	Best fit — no credit card, no storage bill risk, PySpark native

Why Databricks: No billing surprises possible. Compute quota resets daily. Data is never deleted when quota runs out. Perfect for a semester-long academic project.

7.3 Recommended Platform Stack for This Project

Layer	Recommended Platform
Spark Processing	Databricks Free Edition
Orchestration / Scheduling	Databricks Workflows
Raw Data Storage (Bronze)	Databricks Managed Storage
Silver / Gold (Parquet)	Databricks Managed Storage
Version Control	GitHub
Dashboard	Power BI Desktop (free)

7.4 Cost Guardrails & Budget Discipline

The following guardrails are enforced regardless of which platform is used:

1. **Silver transition filter:** The technology universe filter is applied strictly during the Bronze → Silver transition.
2. **Aggressive Bronze retention:** Bronze data is purged after 7 days to eliminate ballooning storage costs, relying on the source archives for any historical backfills.
3. **Date-bounded development:** All development and testing is performed on **7 days of data** maximum. The full 3-month historical backfill runs only once, after the pipeline is validated on the small window.
4. **Parquet + Snappy compression:** Silver and Gold layers use Parquet with Snappy compression, reducing storage footprint by 60–80% versus raw JSON.
5. **Partition pruning:** All Gold queries use `year/month` partition filters. No full-table scans are permitted in production queries.

6. **No always-on clusters:** All Databricks clusters are job-scoped (auto-terminate after job completion). No persistent clusters are left running between runs.
7. **Monthly FinOps review:** At the start of each calendar month, review actual vs estimated consumption in Databricks and adjust batch sizes or frequency if consumption exceeds estimates.

7.5 Estimated Monthly Cost at Scale

Assuming the pipeline runs daily at full operational scale (not development):

Resource	Platform	Estimated Monthly Cost
Spark processing (daily GH Archive ingest)	Databricks Free Edition	\$0 (within fair usage)
Spark processing (daily HF snapshot)	Databricks Free Edition	\$0 (trivially small)
Storage (Bronze + Silver + Gold Parquet)	Databricks Managed Storage	\$0
Version control	GitHub	\$0
Total estimated monthly cost		\$0

This project is **designed to cost nothing** for a full semester when run within Databricks Free Edition + GitHub. The free tier creates a zero-cost full-stack data engineering environment.

8. Definition of Done (Phase 1 Success Criteria)

Phase 1 will be considered complete and successful when the following criteria are met:

1. **Pipeline Execution:** A Databricks Workflow successfully orchestrates the end-to-end pipeline (Bronze → Silver → Gold) for both GH Archive and Hugging Face sources on an automated daily schedule.
2. **Data Model:** The Gold layer contains fully populated, heavily aggregated fact tables (`fact_github_activity`, `fact_hf_model_activity`, `fact_hf_adoption`, `fact_technology_daily_snapshot`) that map back to our curated ~45 anchor technologies.
3. **Dashboard:** A functional BI dashboard is deployed (e.g., via Power BI Desktop or Tableau Public) presenting the 5 core pages outlined in Section 6, querying directly from the Gold Parquet tables.
4. **Data Quality:** The pipeline gracefully handles 429 rate limits from Hugging Face and successfully strips all Hard PII from the GitHub event stream before writing to the Silver layer.
5. **FinOps:** The entire Phase 1 pipeline runs without exceeding the \$0 budget constraints of the Databricks Free Edition and GitHub Student Pack.